`Terraform-06`

1). Watch the **Terraform-06** video.
2). Execute the script shown in the video.

Step 1 — Create your project folders

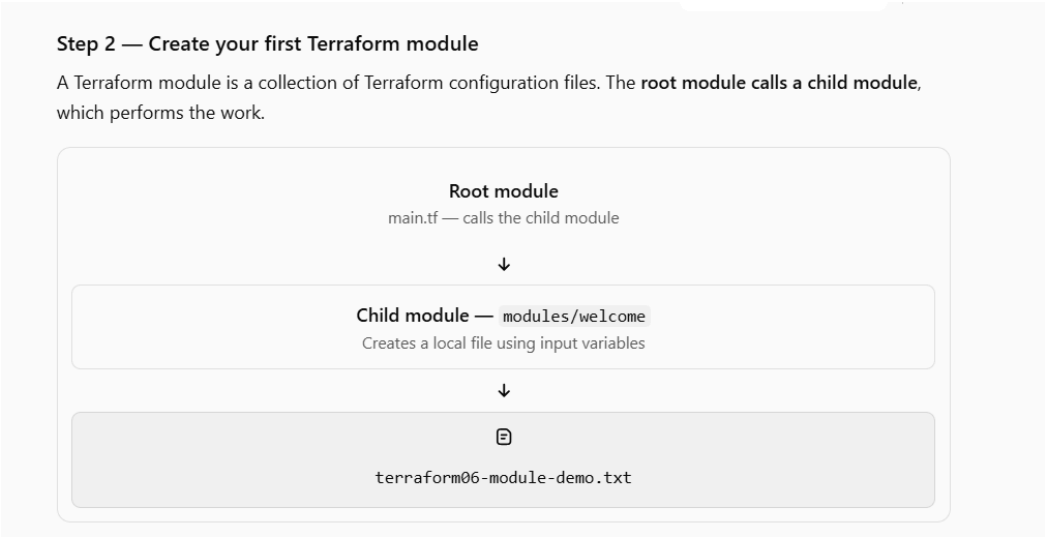
Open Windows PowerShell and copy-paste these commands:

```
$root = "C:\Users\HP\terraform-06"

dirs = @("root\task-2-video-practice",
   "$root\task-3-modules\modules\ec2",
   "$root\task-3-modules\modules\vpc",
   "$root\task-3-modules\modules\s3",
   "$root\task-4-workspaces"
)

$dirs | ForEach-Object {
   New-Item -ItemType Directory -Force -Path $_ | Out-Null
}

Get-ChildItem $root
```



2.1 Create the files

Copy and paste this entire block into your Windows PowerShell terminal:

```
$root = "C:\Users\HP\terraform-06\task-2-video-practice"

New-Item -ItemType Directory -Force -Path "$root\modules\welcome" | Out-Null

@'
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}
'@ | Set-Content "$root\versions.tf"

@'
module "welcome_file" {
  source    = "./modules/welcome"
  file_name = "terraform06-module-demo.txt"
  content   = "Hello! My first Terraform child module is working."
}

output "created_file_path" {
  value = module.welcome_file.file_path
}
'@ | Set-Content "$root\main.tf"

@'
variable "file_name" {
  type = string
}

variable "content" {
  type = string
}
'@ | Set-Content "$root\modules\welcome\variables.tf"
```

```
@'
resource "local_file" "demo" {
  filename = "${path.root}/${var.file_name}"
  content  = var.content
}
'@ | Set-Content "$root\modules\welcome\main.tf"

@'
output "file_path" {
  value = local_file.demo.filename
}
'@ | Set-Content "$root\modules\welcome\outputs.tf"

Set-Location $root
Get-ChildItem -Recurse
```

2.2 Initialize and validate Terraform

Run these commands one by one:

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan
```

Check that validation succeeds and the plan shows 1 resource to add.

2.3 Create and verify the file

```
terraform apply
```

When prompted, enter `yes`.

Then verify:

```
terraform output
Get-Content .\terraform06-module-demo.txt
terraform state list
```

Expected content:

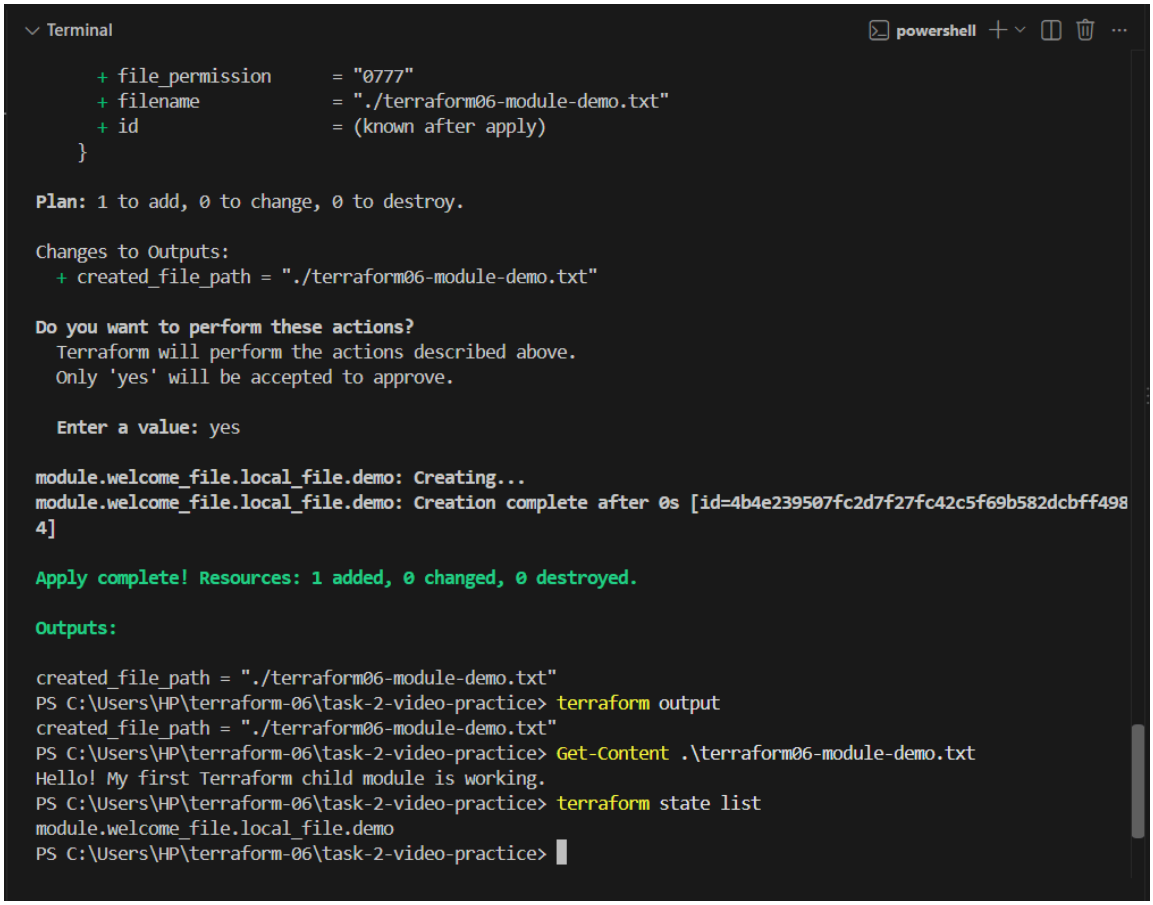
```
Hello! My first Terraform child module is working.
```

Expected resource in the state:

module.welcome_file.local_file.demo

What you learned

- source tells Terraform where the child module is located.
- Input variables pass values from the root module to the child module.
- Outputs return useful information from the child module.
- Terraform manages the resource created inside the module.



3). Provision EC2, S3, and VPC using Terraform modules.

Task 3: Provision EC2, S3, and VPC Using Terraform Modules

1. Objective

The objective of this assignment is to provision AWS infrastructure using reusable Terraform modules.

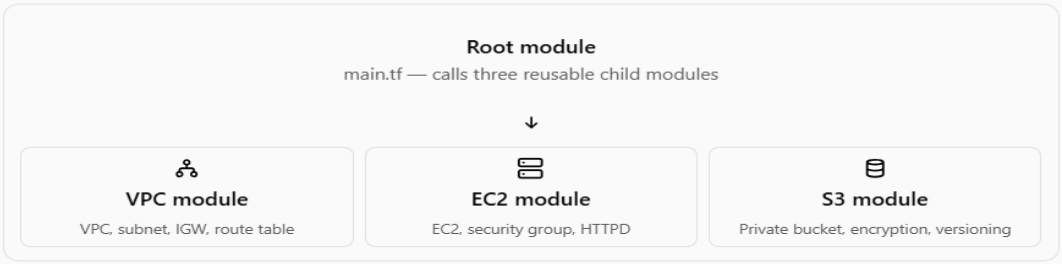
We created three child modules:

1. **VPC module** — Creates the network infrastructure.
2. **EC2 module** — Creates an EC2 web server and its security group.
3. **S3 module** — Creates a secure S3 bucket.

The root Terraform configuration calls these modules and connects them using input variables and output values.

`C:\Users\HP\terraform-06\task-3-modules` folder.

Step 3.1 — Understand the architecture



We'll create a custom VPC and public subnet, launch an EC2 instance with HTTPD installed, and create a private S3 bucket. We'll avoid a NAT Gateway to reduce costs.

Important: EC2 and its public IPv4 address may incur charges. We'll review the plan before creating anything.

2. Tools and Technologies Used

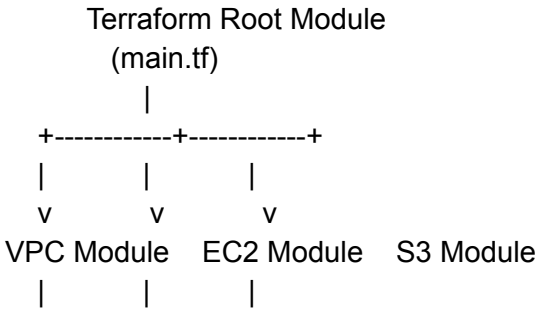
- Terraform
- AWS provider
- AWS CLI
- Amazon EC2
- Amazon VPC
- Amazon S3
- Amazon Linux 2023
- Apache HTTP Server
- Windows PowerShell

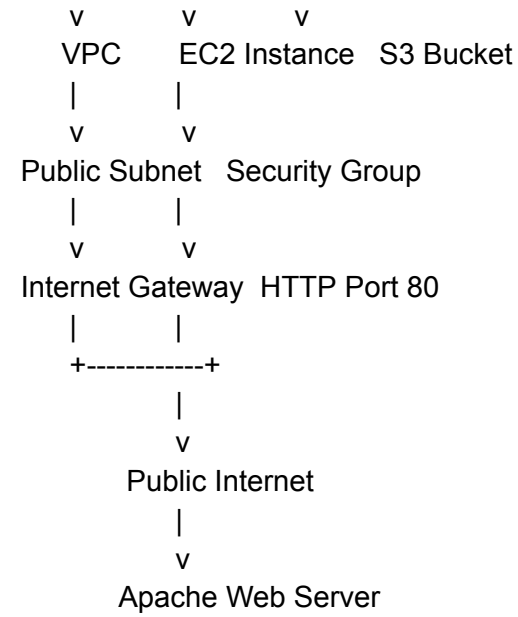
AWS region: `us-east-1`

Terraform project directory:

`C:\Users\HP\terraform-06\task-3-modules`

3. Architecture Diagram





The EC2 instance is deployed inside the public subnet. The Internet Gateway and route table provide internet routing, while the security group permits HTTP traffic on port 80.

The S3 bucket is separate from the web server and has public access blocked, encryption enabled, and versioning enabled.

4. Project Folder Structure

```
terraform-06/
|
+-- task-3-modules/
   |
   +-- versions.tf
   +-- provider.tf
   +-- main.tf
   +-- outputs.tf
   |
   +-- modules/
      |
      +-- ec2/
      |   +-- main.tf
      |   +-- variables.tf
      |   +-- outputs.tf
      |
      +-- vpc/
      |   +-- main.tf
      |   +-- variables.tf
```

```
|   +-- outputs.tf
|
+-- s3/
    +-- main.tf
    +-- variables.tf
    +-- outputs.tf
```

Root module files

- `versions.tf` — Defines the required Terraform version and providers.
- `provider.tf` — Configures the AWS provider, region, and AWS CLI profile.
- `main.tf` — Calls the VPC, EC2, and S3 child modules.
- `outputs.tf` — Displays important resource details after deployment.

Child module files

- `main.tf` — Defines the resources created by the module.
- `variables.tf` — Defines input values received by the module.
- `outputs.tf` — Exposes resource attributes to the root module or other modules.

```
$root = "C:\Users\HP\terraform-06\task-3-modules"


# Root configuration

@'

terraform {

  required_version = ">= 1.5.0"


  required_providers {

    aws = {

      source  = "hashicorp/aws"

      version = "~> 6.0"

    }
```

```
    random = {
      source  = "hashicorp/random"
      version = "~> 3.6"
    }
  }
}
'@ | Set-Content "$root\versions.tf"

@'
provider "aws" {
  region  = "us-east-1"
  profile = "terraform-lab"
}
'@ | Set-Content "$root\provider.tf"

@'
locals {
  common_tags = {
    Project   = "Terraform-06"
    ManagedBy = "Terraform"
  }
}

module "vpc" {
```

```
  source              = "./modules/vpc"
  name                = "terraform06"
  vpc_cidr            = "10.60.0.0/16"
  public_subnet_cidr  = "10.60.1.0/24"
  availability_zone   = "us-east-1a"
  tags                = local.common_tags
}

module "ec2" {
  source        = "./modules/ec2"
  name          = "terraform06-web"
  environment   = "modules-demo"
  instance_type = "t3.micro"
  vpc_id        = module.vpc.vpc_id
  subnet_id     = module.vpc.public_subnet_id
  tags          = local.common_tags
}

module "s3" {
  source = "./modules/s3"
  name   = "terraform06-artifacts"
  tags   = local.common_tags
}
'@ | Set-Content "$root\main.tf"
```

```
@'
output "vpc_id" {
  value = module.vpc.vpc_id
}

output "subnet_id" {
  value = module.vpc.public_subnet_id
}

output "instance_id" {
  value = module.ec2.instance_id
}

output "public_ip" {
  value = module.ec2.public_ip
}

output "website_url" {
  value = module.ec2.website_url
}

output "s3_bucket_name" {
  value = module.s3.bucket_name
```

```
}
'@ | Set-Content "$root\outputs.tf"

# VPC child module
@'
variable "name" {
  type = string
}

variable "vpc_cidr" {
  type = string
}

variable "public_subnet_cidr" {
  type = string
}

variable "availability_zone" {
  type = string
}

variable "tags" {
  type    = map(string)
  default = {}
```

```
}
'@ | Set-Content "$root\modules\vpc\variables.tf"

@'
resource "aws_vpc" "this" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = merge(var.tags, {
    Name = "${var.name}-vpc"
  })
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.this.id
  cidr_block              = var.public_subnet_cidr
  availability_zone       = var.availability_zone
  map_public_ip_on_launch = true

  tags = merge(var.tags, {
    Name = "${var.name}-public-subnet"
  })
}
```

```
resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id

  tags = merge(var.tags, {
    Name = "${var.name}-igw"
  })
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }

  tags = merge(var.tags, {
    Name = "${var.name}-public-rt"
  })
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
```

```
  route_table_id = aws_route_table.public.id
}
'@ | Set-Content "$root\modules\vpc\main.tf"

@'
output "vpc_id" {
  value = aws_vpc.this.id
}

output "public_subnet_id" {
  value = aws_subnet.public.id
}
'@ | Set-Content "$root\modules\vpc\outputs.tf"

# EC2 child module
@'
variable "name" {
  type = string
}

variable "environment" {
  type = string
}
```

```
variable "instance_type" {
  type = string
}

variable "vpc_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "tags" {
  type    = map(string)
  default = {}
}
'@ | Set-Content "$root\modules\ec2\variables.tf"

@'
data "aws_ssm_parameter" "al2023_ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

resource "aws_security_group" "this" {
```

```
  name_prefix = "${var.name}-"
  description = "Allow public HTTP access"
  vpc_id      = var.vpc_id

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(var.tags, {
    Name = "${var.name}-sg"
  })
}
```

```
resource "aws_instance" "this" {
  ami                         = data.aws_ssm_parameter.al2023_ami.value
  instance_type               = var.instance_type
  subnet_id                   = var.subnet_id
  vpc_security_group_ids      = [aws_security_group.this.id]
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    set -eux
    dnf install -y httpd
    systemctl enable --now httpd
    cat > /var/www/html/index.html <<'HTML'
    <h1>Terraform-06 Modules: ${var.environment}</h1>
    <p>EC2 created using a reusable Terraform module.</p>
    HTML
  EOF

  user_data_replace_on_change = true

  root_block_device {
    encrypted   = true
    volume_size = 8
    volume_type = "gp3"
```

```
  }

  tags = merge(var.tags, {
    Name = "$var.name -{var.environment}"
  })
}
'@ | Set-Content "$root\modules\ec2\main.tf"

@'
output "instance_id" {
  value = aws_instance.this.id
}

output "public_ip" {
  value = aws_instance.this.public_ip
}

output "website_url" {
  value = "http://${aws_instance.this.public_ip}"
}
'@ | Set-Content "$root\modules\ec2\outputs.tf"

# S3 child module
@'
```

```
variable "name" {
  type = string
}

variable "tags" {
  type    = map(string)
  default = {}
}
'@ | Set-Content "$root\modules\s3\variables.tf"

@'
data "aws_caller_identity" "current" {}

resource "random_id" "suffix" {
  byte_length = 4
}

resource "aws_s3_bucket" "this" {
  bucket = "var.name −{data.aws_caller_identity.current.account_id}-${random_id.suffix.hex}"

  tags = merge(var.tags, {
    Name = var.name
  })
}
```

```
resource "aws_s3_bucket_public_access_block" "this" {
  bucket                  = aws_s3_bucket.this.id
  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "this" {
  bucket = aws_s3_bucket.this.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "this" {
  bucket = aws_s3_bucket.this.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
```

```
  }
}
'@ | Set-Content "$root\modules\s3\main.tf"


@'
output "bucket_name" {
  value = aws_s3_bucket.this.bucket
}
'@ | Set-Content "$root\modules\s3\outputs.tf"


Set-Location $root
Get-ChildItem -Recurse -File
```

5. Step 1 — Verify AWS Credentials

Open Windows PowerShell and run:

```
aws sts get-caller-identity --profile terraform-lab
```

This command verifies that the configured AWS CLI profile can access the intended AWS account.

Next, navigate to the project directory:

```
Set-Location "C:\Users\HP\terraform-06\task-3-modules"
```

6. Step 2 — Configure the Root Module

The root configuration calls all three child modules.

The VPC module receives the network configuration, including:

- VPC CIDR: `10.60.0.0/16`
- Public subnet CIDR: `10.60.1.0/24`
- Availability Zone: `us-east-1a`

The EC2 module receives the subnet ID and VPC ID from the VPC module outputs.

The S3 module receives the bucket name prefix and common resource tags.

This demonstrates how Terraform modules communicate using input variables and output values.

7. Step 3 — Create the VPC Module

The VPC module provisions:

1. A VPC.
2. A public subnet.
3. An Internet Gateway.
4. A public route table.
5. A route table association.

The route table includes a default route:

```
0.0.0.0/0 → Internet Gateway
```

The public subnet has public IPv4 address assignment enabled.

The VPC module exports the VPC ID and public subnet ID for use by the EC2 module.

8. Step 4 — Create the EC2 Module

The EC2 module provisions an Amazon Linux 2023 EC2 instance and a security group.

EC2 configuration

- Instance type: `t3.micro`
- Operating system: Amazon Linux 2023
- Root volume: 8 GB gp3
- Root volume encryption: Enabled
- Public IP address: Enabled
- Apache HTTP Server: Installed through user data

The Amazon Linux 2023 AMI is retrieved using the AWS Systems Manager public parameter.

Security group configuration

Direction	Protocol	Port	Source
Inbound	TCP	80	`0.0.0.0/0`
Outbound	All	All	`0.0.0.0/0`

No SSH inbound rule is configured.

User data

The EC2 instance runs a startup script that installs Apache, starts the service, enables it at boot, and creates a sample web page.

The page displays:

<h1>Terraform-06 Modules: modules-demo</h1>
EC2 created using a reusable Terraform module.

9. Step 5 — Create the S3 Module

The S3 module creates a uniquely named bucket using the configured prefix, AWS account ID, and a random suffix.

The module configures:

- S3 public access block
- Server-side encryption using AES256
- Bucket versioning

The bucket is not configured for public access.

The S3 module exports the final bucket name.

10. Step 6 — Initialize Terraform

Run:

terraform init

This initializes the working directory, downloads the required providers, and prepares the child modules.

11. Step 7 — Format and Validate the Configuration

Format the Terraform files:

terraform fmt -recursive

Validate the configuration:

terraform validate

Result: `Success! The configuration is valid.`

Validation confirms that Terraform can parse the configuration and that its basic structure is valid.

12. Step 8 — Review the Execution Plan

Run:

terraform plan

Terraform displayed:

Plan: 12 to add, 0 to change, 0 to destroy.

The plan was reviewed before deployment.

The 12 planned resource instances included the EC2 instance, security group, VPC networking resources, S3 bucket and its settings, and the random suffix resource.

13. Step 9 — Apply the Configuration

Run:

terraform apply

Review the proposed changes. Type `yes` when Terraform asks for approval.

Terraform created the resources successfully.

Final result:

Apply complete! Resources: 12 added, 0 changed, 0 destroyed.

14. Step 10 — Verify Terraform Outputs

Run:

terraform output

The successful deployment returned the following values:

Resource	Output
EC2 instance ID	`i-060336d199d6e5990`
EC2 public IP	`3.236.76.51`
VPC ID	`vpc-0b1bd7eb43812687a`
Public subnet ID	`subnet-00e662440d68374e0`
S3 bucket	`terraform06-artifacts-986881030460-d6e17b43`
Website URL	`http://3.236.76.51`

These are the deployment values recorded during the successful apply. The public IP may change if the EC2 instance is recreated.

15. Step 11 — Verify Terraform State

Run:

terraform state list

Terraform displayed the resources using their module paths, including:

module.ec2.aws_instance.this
module.ec2.aws_security_group.this
module.s3.aws_s3_bucket.this
module.s3.aws_s3_bucket_public_access_block.this
module.s3.aws_s3_bucket_server_side_encryption_configuration.this
module.s3.aws_s3_bucket_versioning.this
module.vpc.aws_vpc.this
module.vpc.aws_subnet.public
module.vpc.aws_internet_gateway.this
module.vpc.aws_route_table.public
module.vpc.aws_route_table_association.public

The state list also includes the random ID resource and data sources.

This confirms that Terraform tracks resources created through the child modules.

16. Step 12 — Verify the Website

The website URL was:

`http://3.236.76.51`

To test the web server from PowerShell, run:

```
Invoke-WebRequest -UseBasicParsing `
  -Uri (terraform output -raw website_url) `
  -TimeoutSec 30
```

The verification returned:

```
StatusCode        : 200
StatusDescription : OK
```

The response content was:

```
<h1>Terraform-06 Modules: modules-demo</h1>
<p>EC2 created using a reusable Terraform module.</p>
```

Verification result: Successful. The EC2 instance was serving the expected web page over HTTP.

17. Troubleshooting Notes

Terraform validation fails

Run:

```
terraform fmt -recursive
terraform validate
```

Review any syntax errors reported by Terraform.

Website does not open

Check the following:

- EC2 instance is running.

- The security group permits inbound TCP port 80.
- The subnet route table has a default route through the Internet Gateway.
- Apache has finished installing and starting.
- The instance has a public IP address.

Terraform cannot access AWS

Verify the configured profile:

aws sts get-caller-identity --profile terraform-lab

Confirm that the profile has the necessary permissions.

18. Cleanup and Cost Management

After documenting and verifying the assignment, remove the Task 3 resources if they are no longer required.

First, navigate to the correct project directory:

Set-Location "C:\Users\HP\terraform-06\task-3-modules"

Review the resources that Terraform proposes to delete:

terraform plan -destroy

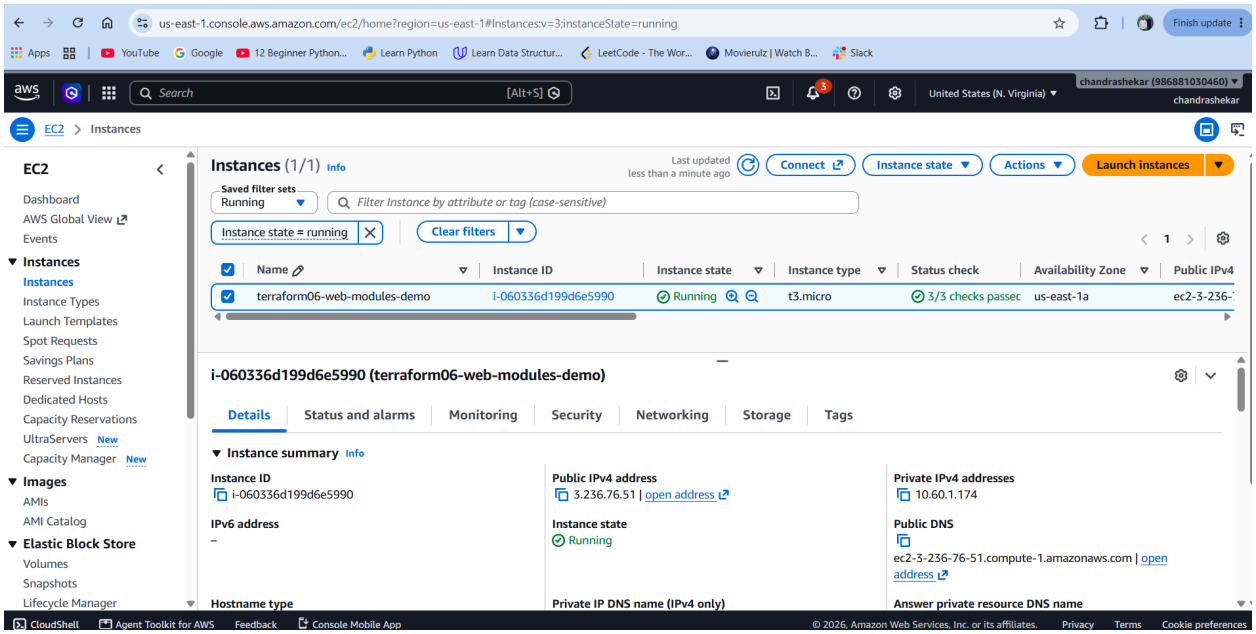
If the plan contains only the Task 3 resources, run:

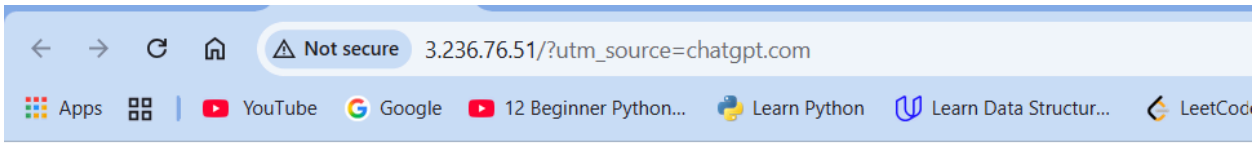
terraform destroy

Review the plan and type `yes` to confirm.

Wait for Terraform to report that the destroy operation completed successfully.

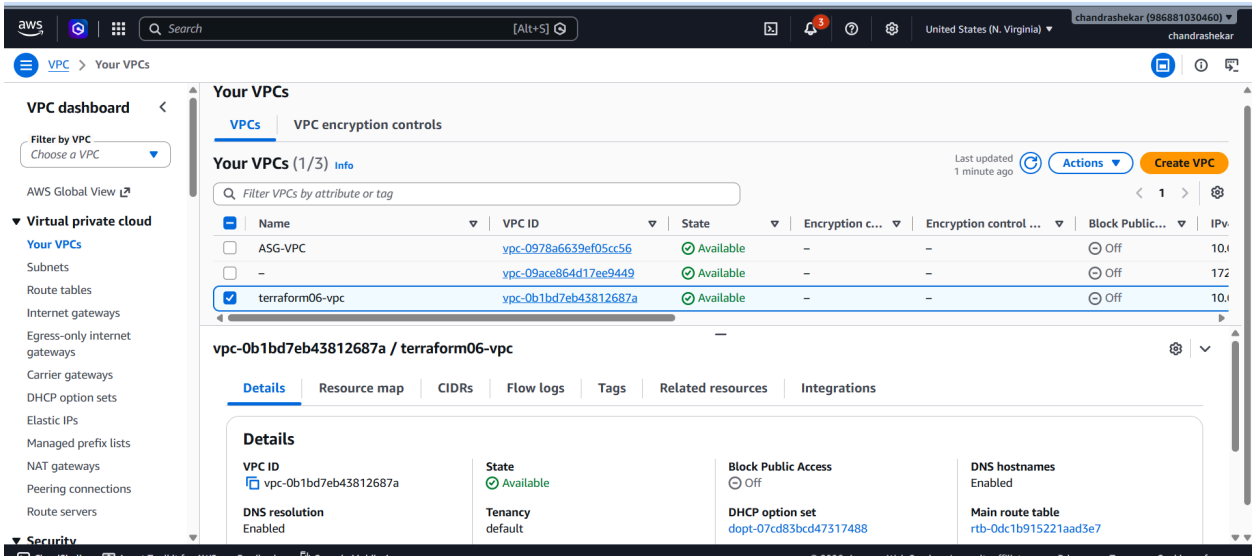
Important: Run the destroy command only from the Task 3 directory. It should not be run from the Terraform-05 backend directory or another project. The Task 3 Terraform files can remain on the computer after cleanup.

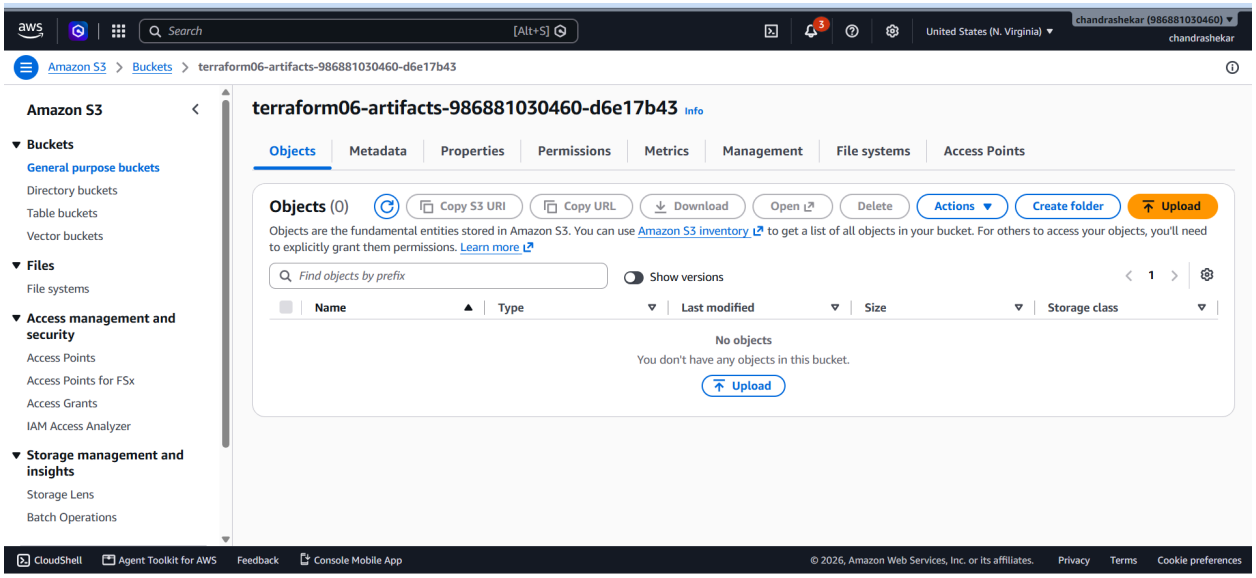




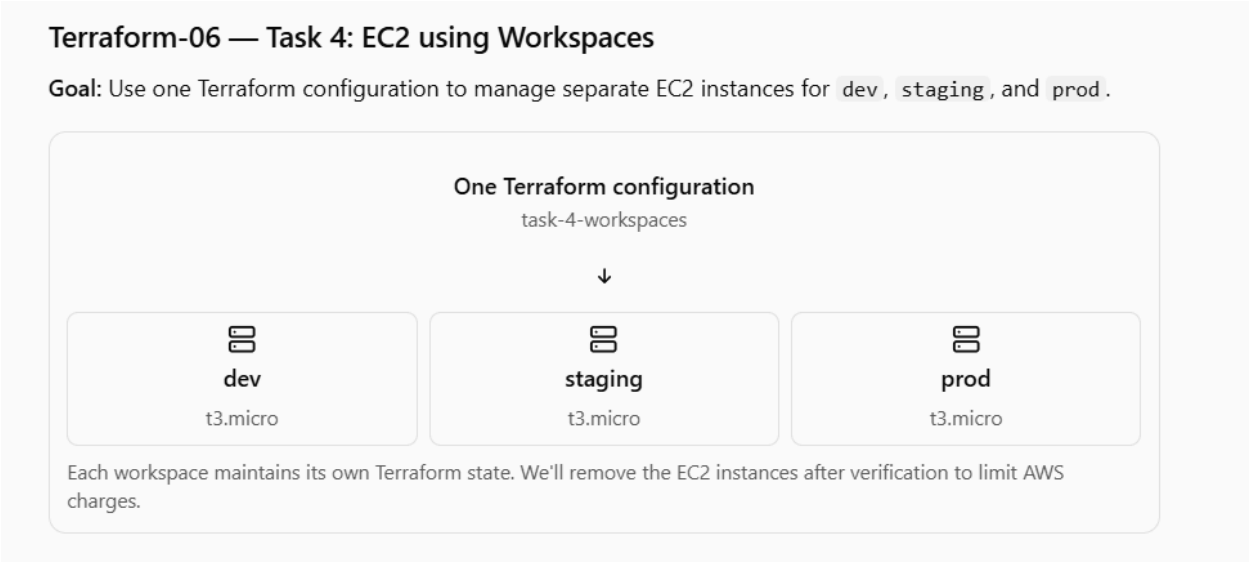
Terraform-06 Modules: modules-demo

EC2 created using a reusable Terraform module.





4). Provision **EC2 for 3 different environments** (Dev, Staging, and Prod) using **Terraform workspaces**.



Provision EC2 Instances Using Terraform Workspaces

1. Objective

The objective of this task was to learn how to use Terraform workspaces to manage separate environments such as Development, Staging, and Production using a single Terraform configuration.

In this practical, we:

- Created Terraform workspaces for Dev, Staging, and Prod.
- Provisioned one EC2 instance in each workspace.
- Used separate Terraform state for each environment.
- Verified the EC2 instances in AWS Console.
- Destroyed the instances after verification to avoid ongoing AWS compute charges.

2. Tools and Technologies

- Terraform
- AWS EC2
- AWS provider
- Amazon Linux 2023 AMI
- AWS Systems Manager Parameter Store
- PowerShell
- AWS Management Console

AWS Region: `us-east-1` (US East — N. Virginia)

EC2 Instance Type: `t3.micro`

3. Project Directory Structure

Project location:

`C:\Users\HP\terraform-06\task-4-workspaces`

Files used:

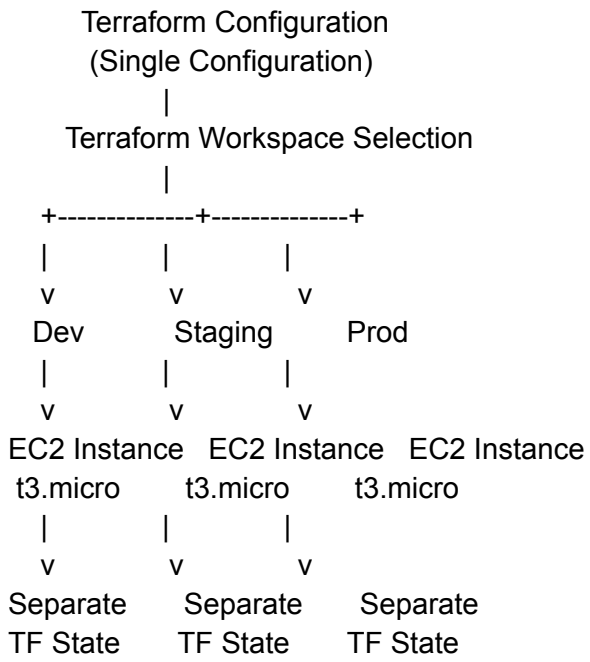
```
task-4-workspaces/
├── versions.tf
├── provider.tf
├── main.tf
├── outputs.tf
└── terraform.tfstate.d/
```

Terraform may create workspace-specific state files under `terraform.tfstate.d` when using local state. The actual state storage depends on the configured backend.

File responsibilities:

- `versions.tf` — Defines the required Terraform version and AWS provider version.
- `provider.tf` — Configures the AWS provider and region.
- `main.tf` — Defines the AMI lookup, default VPC/subnet data sources, workspace-specific configuration, and EC2 resource.
- `outputs.tf` — Displays the workspace name, instance ID, instance type, private IP, and Availability Zone.

4. Architecture



Each workspace maintains its own Terraform state. The same Terraform configuration creates an instance tagged for the selected environment.

Run these commands in PowerShell to create the four files.

1.1 Create `versions.tf`

```
@'

terraform {

  required_version = ">= 1.5.0"



  required_providers {

    aws = {

      source  = "hashicorp/aws"

      version = "~> 6.0"

    }
```

```
  }
}
'@ | Set-Content -Encoding utf8 versions.tf
```

1.2 Create `provider.tf`

```
@'
provider "aws" {
  region  = "us-east-1"
  profile = "terraform-lab"
}
'@ | Set-Content -Encoding utf8 provider.tf
```

1.3 Create `main.tf`

This file defines the three environments and the EC2 resource.

```
@'
data "aws_ssm_parameter" "al2023_ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
```

```
    values = [data.aws_vpc.default.id]
  }
}

locals {
  workspace_config = {
    dev = {
      instance_type = "t3.micro"
    }
    staging = {
      instance_type = "t3.micro"
    }
    prod = {
      instance_type = "t3.micro"
    }
  }

  active_config = lookup(
    local.workspace_config,
    terraform.workspace,
    local.workspace_config["dev"]
  )
}
```

```
resource "aws_instance" "app" {
  count = contains(keys(local.workspace_config), terraform.workspace) ? 1 : 0

  ami                    = data.aws_ssm_parameter.al2023_ami.value
  instance_type          = local.active_config.instance_type
  subnet_id              = data.aws_subnets.default.ids[0]
  associate_public_ip_address = false

  root_block_device {
    encrypted   = true
    volume_size = 8
    volume_type = "gp3"
  }

  tags = {
    Name        = "terraform06-${terraform.workspace}-ec2"
    Environment = terraform.workspace
    Project     = "Terraform-06"
    ManagedBy   = "Terraform"
  }
}
'@ | Set-Content -Encoding utf8 main.tf
```

1.4 Create `outputs.tf`

```
@'
output "workspace" {
  description = "Current Terraform workspace"
  value       = terraform.workspace
}

output "instance_id" {
  description = "EC2 instance ID"
  value       = try(aws_instance.app[0].id, "No EC2 instance in this workspace")
}

output "instance_type" {
  description = "EC2 instance type"
  value       = try(aws_instance.app[0].instance_type, "N/A")
}

output "private_ip" {
  description = "Private IP address"
  value       = try(aws_instance.app[0].private_ip, "N/A")
}

output "availability_zone" {
  description = "EC2 availability zone"
```

```
  value       = try(aws_instance.app[0].availability_zone, "N/A")

}

'@ | Set-Content -Encoding utf8 outputs.tf
```

5. Configure the Terraform Files

Step 1: versions.tf

Configure the minimum Terraform version and AWS provider requirement.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

Step 2: provider.tf

Configure the AWS provider.

```
provider "aws" {
  region  = "us-east-1"
  profile = "terraform-lab"
}
```

The `terraform-lab` AWS CLI profile supplies the credentials used by Terraform.

Step 3: main.tf

The main configuration performs these operations:

1. Retrieves the Amazon Linux 2023 AMI ID from AWS Systems Manager Parameter Store.
2. Retrieves the default VPC and its subnets.
3. Defines the configuration for Dev, Staging, and Prod.
4. Selects the configuration corresponding to the active Terraform workspace.
5. Creates one EC2 instance for the selected environment.

6. Applies environment-specific tags.

The environment configuration uses `t3.micro` for all three environments.

The EC2 resource uses an encrypted 8 GB `gp3` root volume and does not assign a public IP address. It uses the selected subnet in the default VPC.

Important resource settings include:

```
Name        = "terraform06-${terraform.workspace}-ec2"
Environment = terraform.workspace
Project     = "Terraform-06"
ManagedBy   = "Terraform"
```

The configuration uses the workspace name to create the appropriate environment-specific name and tags.

Step 4: outputs.tf

The outputs expose information about the instance created in the active workspace:

- Workspace name
- EC2 instance ID
- Instance type
- Private IP address
- Availability Zone

These outputs make it easier to verify the deployment after applying the configuration.

6. Initialize and Validate Terraform

Open PowerShell and navigate to the project directory.

cd C:\Users\HP\terraform-06\task-4-workspaces

Format the configuration:

terraform fmt

Initialize Terraform:

terraform init

Validate the configuration:

terraform validate

Expected validation result:

Success! The configuration is valid.

7. Create and Deploy the Dev Environment

Create the Dev workspace:

terraform workspace new dev

Review the current workspace:

terraform workspace show

Expected result:

dev

Review the execution plan:

terraform plan

The plan showed one EC2 instance to add.

Apply the configuration:

terraform apply

Review the plan and enter `yes` when prompted.

Verify the deployment:

terraform output
terraform state list

Recorded Dev deployment details:

- Instance ID: `i-0a44b02a10c191b11`
- Instance type: `t3.micro`
- Private IP: `172.31.97.229`
- Availability Zone: `us-east-1b`

8. Create and Deploy the Staging Environment

Create the Staging workspace:

terraform workspace new staging

Check the selected workspace:

terraform workspace show

Generate and review the plan:

terraform plan

Apply the configuration:

terraform apply

Enter `yes` after reviewing the plan.

Verify the resources and outputs:

terraform state list
terraform output

Recorded Staging deployment details:

- Instance ID: `i-078760300d0f4323e`
- Instance type: `t3.micro`
- Private IP: `172.31.97.181`
- Availability Zone: `us-east-1b`

9. Create and Deploy the Prod Environment

Create the Prod workspace:

terraform workspace new prod

Select Prod:

terraform workspace select prod

Check the active workspace:

terraform workspace show

Expected result:

prod

Review the plan:

terraform plan

The plan showed:

Plan: 1 to add, 0 to change, 0 to destroy.

Apply the configuration:

terraform apply

Review the proposed changes and enter `yes`.

Verify the deployed resource:

terraform state list
terraform output

Recorded Prod deployment details:

- Instance ID: `i-0d27d3fb7a2cfffa3`
- Instance type: `t3.micro`
- Private IP: `172.31.97.201`
- Availability Zone: `us-east-1b`

10. Verify All Workspaces

Run:

terraform workspace list

During deployment, the output included:

```
 default
  dev
* prod
  staging
```

The asterisk indicates the currently selected workspace.

To switch environments:

terraform workspace select dev
terraform workspace select staging
terraform workspace select prod

Each environment can be inspected independently using:

terraform workspace show
terraform state list
terraform output

11. Verify in AWS Management Console

Open the AWS EC2 Instances page:

https://console.aws.amazon.com/ec2/home?region=us-east-1#Instances:

Select the `us-east-1` region and inspect the EC2 Instances page.

During deployment, we verified these instances:

Environment	Instance name
Dev	`terraform06-dev-ec2`
Staging	`terraform06-staging-ec2`
Prod	`terraform06-prod-ec2`

All three instances were observed running, with 3/3 status checks passed.

Save the deployment screenshot as assignment evidence.

12. Clean Up the Resources

After completing the deployment and verification, all three EC2 instances were destroyed using Terraform.

Destroy Dev:

terraform workspace select dev
terraform destroy

Review the proposed changes and enter `yes`.

Destroy Staging:

terraform workspace select staging
terraform destroy

Review the proposed changes and enter `yes`.

Destroy Prod:

terraform workspace select prod
terraform destroy

Review the proposed changes and enter `yes`.

All three commands completed successfully with:

Destroy complete! Resources: 1 destroyed.

13. Final Cleanup Verification

The following command confirmed that the workspaces remained available:

terraform workspace list

Then, each workspace was checked independently:

terraform workspace select dev
terraform state list | Select-String "aws_instance"

terraform workspace select staging
terraform state list | Select-String "aws_instance"

terraform workspace select prod
terraform state list | Select-String "aws_instance"

No matching EC2 resource lines were returned for any of the three workspaces.

The workspaces remain available, while their managed EC2 instances have been destroyed. The AWS EC2 Instances page should also be refreshed to confirm the instances are no longer running.

14. What I Learned

- How to create and switch between Terraform workspaces.
- How one Terraform configuration can manage multiple environments.
- How each workspace maintains separate Terraform state.
- How to use terraform plan, terraform apply, terraform output, and terraform state list.
- How to verify provisioned infrastructure in AWS Console.
- How to destroy resources safely after completing a practical assignment.

```
Terminal                                                    powershell

aws_instance.app[0]: Still creating... [00m10s elapsed]
aws_instance.app[0]: Creation complete after 16s [id=i-0d27d3fb7a2cfffa3]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

availability_zone = "us-east-1b"
instance_id = "i-0d27d3fb7a2cfffa3"
instance_type = "t3.micro"
private_ip = "172.31.97.201"
workspace = "prod"
PS C:\Users\HP\terraform-06\task-4-workspaces> terraform workspace show
prod
PS C:\Users\HP\terraform-06\task-4-workspaces> terraform state list
data.aws_ssm_parameter.al2023_ami
data.aws_subnets.default
data.aws_vpc.default
aws_instance.app[0]
PS C:\Users\HP\terraform-06\task-4-workspaces> terraform output
availability_zone = "us-east-1b"
instance_id = "i-0d27d3fb7a2cfffa3"
instance_type = "t3.micro"
private_ip = "172.31.97.201"
workspace = "prod"
PS C:\Users\HP\terraform-06\task-4-workspaces>
PS C:\Users\HP\terraform-06\task-4-workspaces> terraform workspace list
  default
  dev
* prod
  staging

PS C:\Users\HP\terraform-06\task-4-workspaces>
```

